

Outils ton agent par des commandes.

Un serveur déclare son catalogue à chaque message, qu'on s'en serve ou non. Une commande n'entre dans le contexte qu'au moment où l'agent la tape. Voilà comment outiller un agent sans payer le catalogue.

- Ce que coûte un catalogue chargé d'avance.
- La table des commandes, à coller dans tes instructions.
- Un outil maison en une commande, et l'enchaînement en un seul aller-retour.
- Les trois questions d'arbitrage, et les cas où le serveur gagne.
- Les deux nombres qui prouvent qu'un serveur vaut sa place.

Écrit pour quelqu'un qui sait ouvrir un terminal. Les blocs sont à copier tels quels et à adapter à tes outils.

Le catalogue se paie avant la première action.

Anthropic le constatait le 4 novembre 2025 : la plupart des clients MCP chargent toutes les définitions d'outils d'avance, directement dans le contexte. L'agent paie donc le catalogue à chaque message, même s'il n'appelle rien.

Ce qui rend ça invisible

Le coût n'apparaît sur aucune facture d'outil : il se dilue dans la consommation de ton agent, message après message.

Le moment où ça se voit

Quand les conversations deviennent courtes avant d'être pleines, ou quand le coût monte sans que le travail change.

Ce qu'on croit payer

Les appels d'outils effectués

Une fois, au branchement

Ce qu'on utilise

Un outil ajouté sans conséquence

Ce qu'on paie vraiment

Les définitions de tous les outils branchés

À chaque message de la conversation

Ce qu'on a installé un jour et jamais retiré

Un catalogue qui grossit à chaque ajout

L'ordre de grandeur publié

Sur le cas mesuré dans leur article, la consommation passe de 150 000 à 2 000 jetons quand l'agent passe par du code plutôt que par des appels d'outils. Ce chiffre vaut pour ce cas : mesure le tien avant de conclure.

Notre règle depuis le début : quand une ligne de commande existe, l'agent l'utilise. Il tape, il lit, il ne charge rien d'avance.

Une commande, une sortie lue.

Le serveur Déclare ses outils au démarrage, que l'agent s'en serve ou non. Le coût est payé d'avance, à chaque message.	La commande N'existe dans le contexte qu'au moment où l'agent la tape. Ce qui entre en mémoire, c'est sa sortie, et rien d'autre.
---	---

Serveur branché	Commande tapée
Coût fixe à chaque message	Coût au moment de l'usage
Un jeton d'authentification à tenir	L'authentification déjà faite sur la machine
Un processus de plus qui peut tomber	Un programme qui répond ou échoue franchement
Sortie imposée par l'outil	Sortie filtrée par toi, avant lecture

L'objection honnête
MCP a été conçu pour standardiser l'accès aux outils, et il le fait bien. Le reproche porte sur le chargement d'avance, pas sur le protocole lui-même.

L'argument n'est pas idéologique : un serveur qui tombe en pleine session bloque tout, une commande qui échoue renvoie une erreur lisible.

Les commandes, dans tes instructions.

L'agent ne devine pas quels outils existent sur ta machine. Cette table le lui dit, une fois, dans le fichier qu'il lit au démarrage.

Pourquoi une table

Sans liste, l'agent tente des commandes qui n'existent pas sur ta machine, puis abandonne et te demande de faire à sa place.

Ce qu'on n'y met pas

Les commandes destructrices. Elles sont dans le garde-fou, du côté des refus, jamais dans la table des outils.

CLAUDE.md (ou AGENTS.md)

```
# Outils

Ces commandes existent. Tape-les, lis la sortie.
Ne charge aucun catalogue d'avance.

| Besoin          | Commande          |
|-----|-----|
| Code, PR        | gh                |
| Déploiement     | <CLI de ton hébergeur> |
| Base de données | <CLI de ta base>   |
| Tes données     | outils/<ton-script>.sh |

Avant de brancher un serveur : cherche la
commande. Si elle existe, tape-la.
```

Quatre lignes suffisent au départ. Une commande absente de la table ne sera pas utilisée : c'est voulu.

Une commande absente de la table ne sera pas utilisée : c'est le moyen le plus simple de restreindre un agent.

Un outil maison, en une commande.

Tes propres données méritent une commande plutôt qu'un serveur. Un script qui prend un verbe et rend des lignes suffit à outiller un agent.

La règle d'écriture

Un verbe par usage, jamais d'option obscure. L'agent doit deviner l'usage en lisant la première ligne du fichier.

Le test

Lance la commande sans argument : elle doit afficher son usage. C'est ainsi que l'agent apprend à s'en servir seul.

`outils/demandes.sh`

```
#!/bin/bash
# usage : demandes.sh recentes [jours]
#         demandes.sh sans-rappel
# demandes.csv : recue;nom;rappellee
set -euo pipefail
FICHER="${FICHER:-$HOME/data/demandes.csv}"
case "${1:-aide}" in
    recentes)
        j="${2:-7}"
        d=$(date -v-"${j}"d +%F 2>/dev/null || \
            date -d "-$j days" +%F)
        awk -F';' -v d="$d" 'NR==1 || $1 >= d' "$FICHER" ;;
    sans-rappel)
        awk -F';' 'NR==1 || $3 == ""' "$FICHER" ;;
    *)
        echo "usage : $0 recentes [jours] | sans-rappel"
esac
```

Un verbe par usage, une sortie en colonnes. L'agent apprend l'outil en lisant sa ligne d'usage.

Trois questions, un seul aller-retour.

Le vrai gain n'est pas de remplacer un outil par un autre : c'est de faire trois choses en une commande, au lieu de trois échanges qui rechargent le contexte à chaque fois.

Le filtre fait partie de l'outil

Une commande qui rend deux cents lignes coûte deux cents lignes de contexte. Coupe avec head, wc ou une colonne choisie.

Le bon réflexe

Quand l'agent enchaîne toujours les mêmes appels, écris-en la commande. Tu viens de diviser le coût de cette tâche.

outils/chaine.sh

```
# Trois actions, une seule lecture de contexte.

pr=$(gh pr list --state open --json number \
    --jq 'length')
commits=$(git log --since=7.days --oneline | wc -l)
demandes=$(bash outils/demandes.sh sans-rappel \
    | tail -n +2 | wc -l)

printf 'PR ouvertes : %s\n' "$pr"
printf 'Commits sur 7 jours : %s\n' $commits
printf 'Demandes sans rappel : %s\n' $demandes
```

Avant : trois appels d'outil et trois catalogues en mémoire. Après : une commande, trois lignes lues.

Trois questions avant de brancher.

Une seule réponse défavorable suffit à trancher. Écris-les dans une note : c'est ce qui évite d'accumuler des serveurs qu'on n'ose plus retirer.

Le serveur reste le bon choix

Pourquoi

Aucun équivalent en ligne de commande

certains outils métier n'en publient pas

Une authentification complexe

la commande ne sait pas tenir la session

Un standard partagé entre plusieurs clients

le même outil sert à plusieurs agents différents

`regles/arbitrage.md`

CLI ou MCP : trois questions

1. Existe-t-il une commande qui fait la même chose ? Chercher, installer, taper.
2. Combien de jetons ses définitions coûtent-elles à chaque message ? Compter avant.
3. L'agent l'appellera-t-il à chaque tâche, ou une fois par semaine ?

Preuve demandée avant d'installer : le contexte chargé au démarrage, avant et après.

Deux nombres, avant et après.

1

Le contexte au démarrage

Relève-le une fois sans le serveur, une fois avec. La différence est ce que tu payes à chaque message, sans avoir rien appelé.

2

Les appels réels sur une semaine

Compte combien de fois l'agent a vraiment utilisé l'outil. Un serveur branché et appelé deux fois par semaine coûte plus qu'il ne rend.

3

La décision

Si la commande existe et que l'usage est occasionnel, retire le serveur. Sinon, garde-le et note pourquoi dans la note d'arbitrage.

Quand refaire la mesure

À chaque ajout de serveur, et une fois par trimestre. Les catalogues grossissent sans prévenir, au fil des mises à jour.

Le seuil qu'on s'est donné

Un outil appelé moins d'une fois par jour ne reste pas branché en permanence : il devient une commande, ou il sort.

Chez nous

GitHub, l'hébergement, la base de données et les workflows passent par des commandes. Un seul serveur est resté, pour un outil métier qui n'a pas d'équivalent en ligne de commande.

Trois phrases à garder.

- Quand une ligne de commande existe, elle passe devant.
- Une commande n'entre dans le contexte qu'au moment où elle est tapée.
- Le filtre fait partie de l'outil : la sortie, c'est ce que tu payes.
- Un serveur branché « au cas où » se paie à chaque message, sans être appelé une seule fois.

Par où commencer

Liste les serveurs branchés aujourd'hui, et pour chacun, la date du dernier appel réel. Le ménage se fait tout seul.

Le piège

Brancher un serveur « au cas où ».
Il ne sera pas gratuit tant qu'il sera déclaré.

Quand tu veux aller plus loin

Ce que tu ajoutes

Les mêmes trois appels reviennent

une commande qui les enchaîne, une seule sortie

Les sorties sont trop longues

un filtre dans la commande : colonnes, nombre de lignes

Tu ne sais plus ce qui est branché

une note d'arbitrage, une ligne par outil et sa date

Combien de serveurs sont branchés sur ton agent en ce moment ? Réponds à mon message, on regarde lesquels valent leur place.